Laboratorul 3 – Sisteme distribuite

TEMA DE LUCRU

Implementați un sistem distribuit cu minim 3 noduri (rulând pe mașini diferite) care demonstrează funcționarea algoritmului Bully (limbaj Java, consolă, Linux/Ubuntu).

> Analizati exemplele indicate în continuare (Exemplul-1, Exemplul-2) pentru o înțelegere mai buna a algoritmului Bully de alegere a liderului.

Experimentul va consta în
- lansarea succesivă a 3 procese și evidențierea alegerii liderului
- oprirea procesului lider și evidențierea alegerii unui nou lider
- (re)pornirea unui nou proces

Se va urmări și explica schimbul de mesaje dintre procese.

Se pot utiliza mașinile virtuale din laboratorul S3:
192.168.37.219, 192.168.37.237 prin conturile stud1, stud2.. stud5.

Imaginile (capturi ecran) vor afișa pentru fiecare process adresa ip, id-ul și o parte din mesajele afișate la consolă.

Se vor include secvențe de mesaje (text) preluate din consola si se va explica procesul de inițializare, funcționarea, erori/propuneri de perfecționare.

Exemplu

```
Election Initiated..
The process: 192.168.199.253 has FAILED, cannot send Election Request !
Connection established.....
******LEADER selected is :5 : 192.168.199.251 Leader Flag= true Election Request value=
false Value of received= false [heartbeat] destination_server is 192.168.199.251
Sent HEARTBEAT to: 192.168.199.251
[heartbeat] destination_server is 192.168.199.251
Sent HEARTBEAT to: 192.168.199.251
[heartbeat] destination_server is 192.168.199.251
Sent HEARTBEAT to: 192.168.199.251
..
..
### Leader has FAILED! ###
election_req= false received = false
Election Initiated..
Sent Election Request to: 192.168.199.253
The process: 192.168.199.251 has FAILED, cannot send Election Request ! Received OK from
192.168.199.253
******LEADER selected is :2 : 192.168.199.253 Leader Flag= true
```

- lansați în execuție fiecare proces cu un argument indicând ID-ul nodului
- se poate alege id-ul procesului in functie de adresa sa ip, dar sistemul poate avea mai multe interfețe de retea(!)
- pentru transferul unui fisier se poate utiliza comanda **scp**
  `[user]$ scp Bully.class altuser@192.168.199.253:Bully.class`

Exemplul 1

```java
import java.io.*;
import java.util.Scanner;

// create class BullyAlgoExample1 to understand how bully algorithms works
class BullyAlgoExample1 {

  // declare variables and arrays for process and their status
  static int numberOfProcess;
  static int priorities[] = new int[100];
  static int status[] = new int[100];
  static int coord;

  // main() method start
  public static void main(String args[]) throws IOException  // handle IOException
  {
       // get input from the user for the number of processes
       //System.out.println("Enter total number of processes:");
       // create scanner class object to get input from user
       Scanner sc = new Scanner(System.in);
       //numberOfProcess = sc.nextInt();
       numberOfProcess = 5;
       int i;
       // use for loop to set priority and status of each process
       for(i = 1; i<=numberOfProcess; i++)
       {
         //System.out.println("Status for process " + i + ":");
         //status[i] = sc.nextInt();
         status[i]=1;
         if(i==2) status[i]=0;
         //System.out.println("Priority of process " + i + ":");
         //priorities[i] = sc.nextInt();
         priorities[i] = i;
        }

  System.out.println("Enter proces which will initiate election");
  int ele = sc.nextInt();
  sc.close();

  if(status[ele]==0){
      System.out.println("Sorry, process " + ele + " is down.");
      System.exit(1);
  }

  // call electProcess() method
  electProcess(ele);
  System.out.println("After electing process the final coordinator is " + coord);
}

// create electProcess() method
static void electProcess(int ele)
{
  coord = ele;
  //System.out.println("I'm " + ele + " with priority " + priorities[ele]);
  for(int i = 1; i<=numberOfProcess; i++)
  {
      if(priorities[ele] < priorities[i])
      {
          System.out.println("Election message is sent from "+ ele + " to " + i );
          if(status[i]==1)
             electProcess(i);
      }

   }

 }
```

Exemplul 2

```
import java.util.Scanner;
// create process class for creating a process having id and status
class Process {
    public int id;
    public String status;
    public Process(int id){
        this.id = id;
        this.status = "active";
    }
}


public class BullyAlgoExample2 {
    Scanner sc;
    Process[] processes;
    int n;
    public BullyAlgoExample2(){
        sc= new Scanner(System.in);
    }
    // method for initializing the processes
    public void initialize(){
        // get input from the user for processes
        System.out.println("Enter total number of processes of Processes");
        n = sc.nextInt();
        // initialize processes array
        processes = new Process[n];
        for(int i = 0; i<n; i++){
            processes[i]= new Process(i);
        }
}

    // create election() method for electing process
    public void performElection(){
        // we use the sleep() method to stop the execution of the current thread
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        // show failed process
        System.out.println("Process with id " + processes[getMaxValue()].id + " fails");
        // change status to Inactive of the failed process
        processes[getMaxValue()].status = "Inactive";
        // declare and initialize variables
        int idOfInitiator = 0;
        boolean overStatus = true;
        // use while loop to repeat steps
        while(overStatus){
            boolean higherProcesses = false;
            // iterate all the processes
            for(int i = idOfInitiator + 1; i< n; i++){
                if(processes[i].status == "active"){
                    System.out.println("Process " + idOfInitiator
                           + " Passes Election("
                           + idOfInitiator+") message to process" +i);
                    higherProcesses = true;
                 }
             }
              // check for higher process
            if(higherProcesses){
                // use for loop to again iterate processes
                for(int i = idOfInitiator + 1; i< n; i++){
                    if(processes[i].status == "active"){
                        System.out.println("Process “ + i
                           + " Passes Ok("+i+") message to process" + idOfInitiator);
                     }
                }
```

```java
                // increment initiator id
                idOfInitiator++;
            }
            else{
                // get the last process from the processes that will become coordinator
                int coord = processes[getMaxValue()].id;
                // show process that becomes the coordinator
                System.out.println("Finally Process " + coord + " Becomes Coordinator");
                for(int i = coord - 1; i>= 0; i--){
                    if(processes[i].status == "active"){
                        System.out.println("Process " + coord
                           + " send Coordinator(" + coord + ") message to process " +i);
                   } }

                System.out.println("End of Election");
                overStatus = false;
                break;
            }
    // create getMaxValue() method that returns index of max process
    public int getMaxValue(){
        int mxId = -99;
        int mxIdIndex = 0;
        for(int i = 0; i<processes.length; i++){
            if(processes[i].status == "active" && processes[i].id >mxId){
                mxId = processes[i].id;
                mxIdIndex = i;
            }
        }
        return mxIdIndex;
    }

    // main() method start
    public static void main(String[] args) {
       BullyAlgoExample2 bully = new BullyAlgoExample2();
       bully.initialize();
       bully.performElection();
    }

}
```

In exemplul următor se exemplifica posibilitatea de a lansa un proces din Java, în acest caz se lansează o comanda de listare a fișierelor din directorul curent:

```
package ro.cunbm;

import java.io.*;
import java.util.concurrent.*;
import java.util.function.Consumer;

public class Test {

  private static class StreamGobbler implements Runnable {
    private InputStream inputStream;
    private Consumer<String> consumer;

    public StreamGobbler(InputStream inputStream, Consumer<String> consumer) {
      this.inputStream = inputStream;
      this.consumer = consumer;
    }

    @Override
    public void run() {
      new BufferedReader(new InputStreamReader(inputStream)).lines()
        .forEach(consumer);
    }

  }


  public static void main(String args[])
    throws InterruptedException, IOException, ExecutionException, TimeoutException {

    boolean isWindows = System.getProperty("os.name")
      .toLowerCase().startsWith("windows");

    ProcessBuilder builder = new ProcessBuilder();
    if (isWindows) {
      builder.command("cmd.exe", "/c", "dir");
    } else {
      builder.command("sh", "-c", "ls");
    }

    builder.directory(new File(System.getProperty("user.home")));
    Process process = builder.start();
    StreamGobbler streamGobbler =
    new StreamGobbler(process.getInputStream(), System.out::println);
    Future<?> future = Executors.newSingleThreadExecutor().submit(streamGobbler);
    int exitCode = process.waitFor();
    assert exitCode == 0;
    future.get(10, TimeUnit.SECONDS);
  }
}
```